VoiceForm AI — MVP Implementation Specification

> **Purpose:** Build a voice-first form application where a respondent can answer questions naturally by speaking. Speech is transcribed, an AI model interprets the answer in the context of the current question, produces a clean/structured answer, and automatically fills the form.

Target outcome: A working end-to-end MVP that can be run locally and deployed to Vercel. The MVP must prioritize the core voice → AI → form-fill loop over advanced integrations.

1. Product Vision

The product is an AI-powered alternative to traditional typing-based forms. A form creator creates questions; a respondent opens a public form, listens to or reads each question, speaks their answer, and the system converts natural speech into an appropriate form response.

> Core value proposition: **Speak naturally; the AI turns the response into clean form data.**

2. MVP Scope

Must have	Description
Form creation	Create a form with title, description, questions, question type, options where applicable.
Public respondent view	Open a form through a shareable/public URL.
Voice capture	Browser microphone recording with clear recording/stop states.
Speech-to-text	Convert recorded speech to text using an appropriate OpenAI speech model/API.
Question-aware AI	Send question, field type/options, and transcript to an LLM; return a structured answer.
Automatic filling	Populate the current response field with the AI result.
Confirmation	Show the interpreted answer and allow the respondent to accept, retry, or edit.
Field types	Short text, long text, yes/no, multiple choice, rating 1–5, number.
Progression	Next/previous question and progress indicator.
Submission	Save completed responses to the database.
Creator results	Creator can view submitted responses in a simple table.

Explicitly out of scope for MVP: Google Forms import/sync, payments, teams/roles, advanced analytics, multilingual voice optimization, complex branching logic, custom domains, and production-grade enterprise security/compliance.

3. Recommended Tech Stack

Frontend: Next.js + TypeScript + React + Tailwind CSS. Use a clean responsive UI. **Backend:** Next.js server/API routes. **Database/Auth:** Supabase PostgreSQL + Supabase Auth if authentication is needed for creator pages. **AI:** OpenAI APIs for speech-to-text and LLM processing. **Deployment:** Vercel. Keep provider-specific code isolated so models can be swapped later.

Do not over-engineer the MVP. Prefer simple server actions/API routes, typed interfaces, reusable components, and a small number of database tables.

4. Core User Flows

Creator: Sign in → Dashboard → Create form → Add questions → Save → Copy public link → View responses.

Respondent: Open public form → See question → Press microphone → Speak → Stop → Transcribe → AI interprets → Preview answer → Accept/edit/retry → Next question → Submit.

Conversational enhancement: Do not make this mandatory for the first implementation. Structure the code so a later version can let the AI ask a follow-up when an answer is ambiguous or incomplete.

5. UI Requirements

Creator dashboard: clean card/list layout showing forms, response count, create form button, and open/view results actions.

Form builder: title + description; question list; add/edit/delete/reorder questions; type selector; options editor for multiple choice; required toggle; save/publish.

Respondent screen: one question at a time; question text prominent; microphone button; recording animation/state; transcript/AI answer preview; edit field; retry; next/back; progress; final submit.

Visual style: modern, minimal SaaS UI. Avoid unnecessary gradients/animations. Make the microphone interaction obvious. Mobile responsive.

6. Data Model

```
forms
- id (uuid, primary key)
- title
- description
- created_by
- is_published
- public_slug
- created_at
- updated_at

questions
- id (uuid, primary key)
- form_id (foreign key)
- question_text
- question_type
- options (jsonb, nullable)
- is_required
- position
- created_at

sessions
- id (uuid, primary key)
- form_id
- started_at
- completed_at
- status

responses
- id (uuid, primary key)
- session_id
- question_id
- raw_transcript
- ai_answer (jsonb/text)
- confidence (numeric, nullable)
- was_edited (boolean)
```

```
- created_at
```

Use UUIDs. Add indexes for form_id, public_slug, session_id, and question_id. Enforce foreign keys. Store structured answers in a predictable JSON shape where useful.

7. Question Types and AI Output

Type	Example	Expected AI output
short_text	What is your job?	A concise string
long_text	What did you like?	Clean paragraph
yes_no	Would you recommend us?	Yes or No
multiple_choice	Which role?	Exactly one provided option
rating	Rate 1–5	Integer 1–5
number	How many years?	Numeric value

8. AI Processing Contract

The AI must receive at least: current question text, question type, allowed options when applicable, transcript, and whether the question is required. It must never invent information. It must select only from supplied options for multiple-choice fields and stay within valid numeric/rating ranges.

Recommended response contract:

```
{
"answer": "Software Developer",
"confidence": 0.98,
"needs_confirmation": false,
"reason": "The user explicitly identified their occupation."
}
```

For uncertain or conflicting responses, return needs_confirmation=true. Never silently submit a low-confidence interpretation.

9. AI Prompt

```
SYSTEM:
You are a form-response extraction assistant. Interpret the user's spoken response only in the
context of the supplied question. Preserve the user's meaning, remove filler and repetition, and
never invent facts.

Rules:
1. Answer only the current question.
2. For multiple choice, select exactly one option from the supplied options.
3. For yes/no, return Yes or No when the meaning is clear.
4. For rating, return an integer within the allowed range.
5. For number fields, extract the number when clearly stated.
6. If the response is ambiguous, insufficient, or conflicts with the available choices, set
needs_confirmation=true.
7. Do not add information that the user did not say.
8. Return strict JSON matching the requested schema.
```

10. API / Server Design

Endpoint	Purpose
POST /api/forms	Create a form
GET /api/forms/[id]	Get creator form
PATCH /api/forms/[id]	Update form
POST /api/forms/[id]/publish	Publish form
GET /api/public/forms/[slug]	Get public form without private creator data
POST /api/speech/transcribe	Accept audio and return transcript
POST /api/ai/process-answer	Process transcript against question
POST /api/sessions	Create respondent session
POST /api/responses	Save a response
POST /api/sessions/[id]/submit	Complete submission
GET /api/forms/[id]/responses	Creator results

Audio should be handled securely and should not be persisted permanently unless needed. Use server-side environment variables for API keys. Never expose the OpenAI API key to the browser.

11. Error Handling

Handle microphone permission denial, no microphone, recording failure, speech-to-text failure, empty transcript, AI timeout/error, invalid AI JSON, low confidence, unsupported browser features, network loss, and form submission errors. Always provide a retry path and preserve already completed answers where possible.

12. Security Basics

Validate all input server-side. Do not trust client-provided question types/options. Ensure public form endpoints expose only published form content. Protect creator routes with authentication. Keep secrets in environment variables. Add reasonable request/body size limits for audio. Do not log raw sensitive transcripts in production logs.

13. Suggested Project Structure

```
app/
dashboard/
forms/
f/[slug]/
api/
forms/
public/
speech/
ai/
sessions/
responses/
components/
form-builder/
voice-recorder/
question-renderer/
response-preview/
ui/
lib/
ai/
speech/
supabase/
validation/
types/
supabase/
migrations/
```

Keep AI and speech provider implementations under lib/ so the UI does not directly depend on provider SDK details.

14. Implementation Plan

1. Initialize Next.js/TypeScript/Tailwind project and install only necessary dependencies.

2. Configure environment variables and Supabase client/server utilities.

3. Create database schema/migrations and seed one demo form.

4. Build creator dashboard and form builder.

5. Build public respondent form and question renderer.

6. Implement browser microphone recording.

7. Implement speech-to-text API route.

8. Implement question-aware AI processing with strict JSON validation.

9. Connect AI output to response fields with confirmation/edit/retry.

10. Implement sessions, response persistence, and final submission.

11. Build creator response table.

12. Add validation, loading/error states, responsive UI, and basic security checks.

13. Run typecheck, lint, build, and an end-to-end manual test.

15. Acceptance Criteria

✓ A creator can create and publish a form containing at least all MVP question types.

✓ A respondent can open a public URL without creator access.

✓ The browser can record speech after microphone permission.

✓ Speech is converted to a transcript successfully.

✓ The AI receives the question context and returns validated structured output.

✓ The correct field is populated automatically.

✓ The respondent can edit, retry, or confirm the answer.

✓ Low-confidence/ambiguous answers require confirmation instead of silent submission.

✓ All completed responses persist in Supabase.

✓ The creator can view submitted responses.

✓ Refreshing or navigating should not destroy already-saved completed responses.

✓ The application builds successfully and has no TypeScript errors.

16. Demo Scenario

```
Create a demo form called “SaaS Customer Feedback” with:
1. What is your role? — multiple choice
2. How long have you used our product? — number
3. What do you like most about it? — long text
4. What could we improve? — long text
5. How satisfied are you from 1 to 5? — rating
6. Would you recommend it to a colleague? — yes/no

The ideal demo is a user answering these questions naturally by voice while the UI visibly fills the
responses.
```

17. Future Roadmap (Do Not Implement in MVP)

Google Forms import/sync; Google Sheets export; automatic form generation from a natural-language description; multilingual speech; Urdu/Roman Urdu support; conversational follow-up questions; conditional branching; analytics; team collaboration; custom branding; integrations with Typeform/Notion/Airtable; respondent authentication; advanced anti-abuse/rate limiting.

18. Instructions for the Coding Agent

> **Do not stop at generating a plan.** Read this specification and implement the working MVP in the existing repository. Inspect the repository first and preserve useful existing code. If the repository is empty, initialize the project. Make reasonable decisions without repeatedly asking for confirmation. Use placeholders only when a real implementation is impossible without credentials, and clearly document the required environment variables. After implementation, run typecheck/lint/build and fix errors.

The coding agent should prioritize a working end-to-end vertical slice over visual perfection: create form → publish → respondent speaks → transcript → AI structured answer → confirmation → save response → creator views response.